S P E C I F I C A T I O N

Behavioral Specification Cards

Step 2 of 5 — Reverse-Engineered Runtime Behavior of 5 Reference Repositories

Project: Guest Profiling System · Repo: lordwhitefire/guest-profiling-system

Phase: 2 of 5 — Reverse Engineering

Method: Source-reading only (no code copied); stack-agnostic behavioral cards

Prepared by: Z.ai Builder Agent

Date: 2026-11-10

1. Executive Summary

This document contains the reverse-engineered runtime behavior of the five open-source repositories identified in the Discovery Report (Step #1). Each repository is captured as a single behavioral specification card: a stack-agnostic description of what the system does at runtime, expressed in terms of inputs, outputs, state, latency, edge cases, and error behavior. No source code has been copied. Each card is precise enough to re-implement the behavior in any language or framework we choose for the Guest Profiling System.

The five cards together cover every runtime behavior the Guest Profiling System must reproduce. Card 1 (SSLR) is the read-side sync primitive. Card 2 (AWS Sample Hospitality) is the clean guest API contract and the tenant-isolation pattern. Card 3 (KevAlien CRM) is the hospitality-specific guest lifecycle state machine. Card 4 (QloApps) is the messy source PMS — the shape of data our adapter will encounter in real hotels. Card 5 (FormFilla) is the smart form-fill behavior the browser-extension adapter must reproduce on the hotel's existing registration form.

Method note: SSLR was reverse-engineered by reading its 1,614 lines of Go source in full; the behavioral card reflects the actual code, not just the README. AWS Sample Hospitality was reverse-engineered by reading the Python Lambda handlers and the SQL migrations that define the guest schema. KevAlien CRM was reverse-engineered by reading the LangGraph state graph and the scheduler timeline. QloApps was reverse-engineered by reading the MySQL schema dump and the customer search SQL. FormFilla was reverse-engineered by reading the matching module (scorer, assignment, native setter) and the dictionary JSON. None of the five repositories was executed; all behavioral claims come from source reading.

2. Method and License Inventory

Before presenting the spec cards, the table below records each repository's license so that the downstream Product-Manager and OpenSpec steps can make informed code-reuse decisions. Behavioral study is permitted under all four licenses shown; code reuse is permitted only for MIT and Apache-licensed code, and is forbidden for code carrying the Commons Clause.

Table 1: License inventory for the five reference repositories

Repository	License	Permits behavioral study	Permits code reuse
erkkah/sslr	MIT	Yes	Yes
aws-samples/sample-hospitality-systems	AWS Sample Code (modified Apache)	Yes	Yes (with attribution, per AWS Sample Code terms)
KevAlien/CRM	MIT + Commons Clause	Yes	NO — Commons Clause forbids commercial sale of the software
qloapps/qloapps	OSL-3.0 (core), AFL-3.0 (modules)	Yes	Yes (OSL-3.0 is a weak copyleft; modifications to original files must be re-published under OSL-3.0)
AhmadHassan-BTed/FormFilla	MIT	Yes	Yes

License conclusion for the Guest Profiling System: we will re-implement all five behaviors from scratch in our own code (TypeScript / Node.js for the core and adapters, TypeScript / React for the frontend). We will not import or fork any source code from any of the five repositories. The behavioral cards in this document are the only thing carried forward into Step #3 (PM) and Step #4 (OpenSpec).

3. Spec Card 1 — SSLR (Read-Only PostgreSQL Replication)

Source: github.com/erkkah/sslr · License: MIT · Stack observed: Go (pgx driver) · LOC: 1,614

Behavioral summary: SSLR is a one-way, throttled, filtered replication tool that keeps a target PostgreSQL database synchronized with a source PostgreSQL database using only a read-only connection to the source. No server-side extensions, no logical replication slot, and no superuser privileges are required on the source. The contract is exactly what the PMS Adapter must satisfy when a hotel grants us a read-only database login against their PMS database.

3.1 Inputs

SSLR accepts a single JSON configuration file (default sslr.json; override with the -cfg flag). The required fields are source and target connection URLs, plus a list of table names. Everything else is optional with sensible defaults.

Table 2: SSLR configuration inputs (runtime contract)

Field	Type	Required	Default	Runtime meaning
source	string (URL)	Yes	—	PostgreSQL connection URL. Must be read-only; the recommended pattern is to append ?default_transaction_read_only=true
target	string (URL)	Yes	—	PostgreSQL connection URL. Must be writable; SSLR creates the __sslr_state table here
tables	array<string>	Yes	—	List of source table names to sync. Must have a primary key
filteredTables	map	No	{}	Per-table WHERE clause filtering. Each entry has where (string), wheres (array<string>, joined with space), uses (array<string>)
updateChunkSize	uint32	No	1000	Number of rows fetched per source read round in update phase
deleteChunkSize	uint32	No	1000	Initial chunk size for delete-detection scan
minDeleteChunkSize	uint32	No	100	Floor for divide-and-conquer; if hashes still differ at this size, the range is re-synced directly
throttlePercentage	float64	No	80	Max source-DB utilization as % of total execution time. Clamped to [1, 100]
stateTable	string	No	"__sslr_state"	Name of the replication-cursor table in the target DB
syncUpdates	bool	No	true	Whether to sync inserts and updates
syncDeletes	bool	No	true	Whether to scan for and apply deletes
resyncOnSchemaChange	bool	No	false	If true, perform a full table copy on schema drift; if false, abort the job
fullCopyThreshold	float64	No	0.5	If target row count < (fullCopyThreshold × source row count), do a full copy instead of incremental
waitBetweenJobs	duration	No	5s	Pause between sync passes in -c (continuous) mode

Command-line flags: -cfg <path> sets the config file path; -c enables continuous mode (otherwise the tool runs a single sync pass and exits).

3.2 Outputs and state

SSLR's only output is the target database itself. There is no API, no event stream, and no notification mechanism. The tool's progress is persisted in a single state table, which has the following schema (auto-created on first run):

Table 3: SSLR state table schema (created in target DB)

Column	Type	Purpose
table_name	varchar(512) PRIMARY KEY	Name of the synced source table
last_seen_xmin	bigint	Highest xmin value successfully synced to target. The replication cursor
where_clause	varchar	The WHERE clause currently in effect for filtered tables. Changing it triggers a re-sync if resyncOnSchemaChange=true

Stdout logs are emitted at INFO level by default; setting LOG_LEVEL=debug enables per-chunk logging of source and target hash comparisons, throttle utilization, and chunk boundaries.

3.3 Runtime algorithm (observed from source)

On each sync pass the Job.Run() method executes the following sequence, table by table (no two tables run in parallel within a single process; parallelism requires separate SSLR processes with disjoint table lists):

- Connect. Open two pgx connections, one to source (read-only) and one to target (writable). No connection pooling — single connections only.
- Validate tables. For each table: extract schema (CREATE TABLE statement) from source, check if target table exists, create it if missing, compare schemas if present. On schema mismatch: abort unless resyncOnSchemaChange=true, in which case drop+recreate the target table and mark for full sync.
- Extract indices. Pull the source table's primary-key definition and indices. Apply indices to target. Sort PK columns alphabetically (used for deterministic range queries in delete detection).
- Update phase (if syncUpdates). Read state.last_seen_xmin. If 0 or forceSync is set, do a full table copy. Otherwise, query source for the current highest xmin, then walk from (last_seen_xmin + 1) to current max in chunks of updateChunkSize rows.
- Delete phase (if syncDeletes). Get the source PK range (min + count). Walk that range in chunks of deleteChunkSize. For each chunk, hash the PK list on source and on target (md5 of array_agg of PK::varchar). If hashes match, no change. If

they differ and chunk size > minDeleteChunkSize, halve and recurse. If they differ at minDeleteChunkSize, re-sync that exact range.

- Persist state. After each chunk's transaction commits, write the new last_seen_xmin back to the state table.

3.4 Update algorithm — xmin tracking

PostgreSQL assigns every row an xmin system column equal to the transaction ID that last inserted or updated it. SSLR exploits this to detect changes without server-side triggers or logical replication. The source query is:

```
SELECT xmin, * FROM {table} WHERE xmin::text::bigint >= $1  --
last_seen_xmin + 1   [AND {filter_where_clause}] ORDER BY
xmin::text::bigint ASC, {pk_columns} ASC OFFSET $2 LIMIT $3  --
chunk size
```

Critical detail: a transaction with row count > updateChunkSize is fetched across multiple chunks, but the state.last_seen_xmin is only advanced past a transaction when ALL of that transaction's rows have been read. SSLR detects this by tracking when the xmin value changes between consecutive rows — until it changes, the offset counter increments and the cursor stays put. Only once a complete transaction is consumed does the state advance.

Apply step: each chunk's rows are written to the target in a single transaction. Within that transaction, SSLR first deletes any rows whose PK matches the chunk's rows (DELETE … WHERE (pk_cols) IN (SELECT * FROM unnest($1::type[]))) and then re-inserts them via pgx.CopyFrom. This is an idempotent upsert — running the same chunk twice produces the same target state.

3.5 Delete algorithm — divide and conquer

Deletes are the expensive case. SSLR cannot use xmin to detect deletes because a deleted row is gone from the source. Instead, it scans PK ranges and compares hashes. The algorithm is recursive:

- Get the source table's PK range: SELECT {pk_cols}, (SELECT count(*) FROM {table}) FROM {table} ORDER BY {pk_cols} ASC LIMIT 1. This returns the minimum PK and the total row count.
- Start scanning from the minimum PK in chunks of deleteChunkSize.
- For each chunk [startKey, endKey]: query the source for SELECT coalesce(md5(array_agg((pk_cols)::varchar)::varchar), '') FROM {table} WHERE pk_cols in [startKey, endKey]. Do the same on the target.

- If source_hash == target_hash, the range is identical; move on to the next chunk (startKey = endKey).
- If they differ and chunkSize > minDeleteChunkSize: recurse with chunkSize/2, starting at startKey, then recurse again starting at the midpoint. This is classic divide-and-conquer.
- If they differ and chunkSize <= minDeleteChunkSize: stop splitting. The range [startKey, endKey] is small enough to re-sync directly. In a single transaction: DELETE FROM {table} WHERE pk in [startKey, endKey]; then CopyFrom all source rows in [startKey, endKey].

3.6 Throttling

Throttle is implemented as a simple time-budget governor. For each chunk operation (read+apply), SSLR records start time, end time, and accumulates totalJobDuration (time spent in source DB operations). At the end of each chunk, it computes utilization = totalJobDuration / totalDuration and sleeps for 2 × (utilization - target_level) milliseconds if utilization exceeds the configured throttlePercentage. The factor of 2 is intentional — it brings the running average down to the target over the next chunk. Throttle is bypassed entirely if throttlePercentage >= 100.

3.7 Edge cases and failure modes (observed in code)

Table 4: SSLR edge-case behavior

Trigger	Observed behavior
Source table has no primary key	Job aborts with: "table %v does not have a primary key"
Source schema changes (column added/removed)	If resyncOnSchemaChange=false (default): job aborts with errSchemaMismatch. If true: target table is dropped and recreated; forceSync flag set; full copy on next sync pass
Filtered table WHERE clause changes between runs	Same as schema change: abort or full re-sync depending on resyncOnSchemaChange
Source connection drops mid-pass	Job returns error and exits (exit code 3). State table retains last successful xmin; next run resumes from there
Target row count drifts below (fullCopyThreshold × source count)	Full table copy is performed instead of incremental. Triggered when target has lost >50% of rows (default 0.5)
xmin wraps (PostgreSQL xid wraparound)	NOT HANDLED — documented as a known issue. Likely to cause silent replication breakage on long-running source DBs

Trigger	Observed behavior
Same table synced by two SSLR jobs in parallel	UNDEFINED BEHAVIOR — both jobs read+write the same state row and the same target rows; no locking. Documented constraint: same table must be synced by at most one job
Filtered table depends on another filtered table that has not yet been synced	Detected during validation; job aborts with "unknown table %q in uses list"
Row contains non-integer PK	Supported. PK type is detected at runtime; deleteRows uses varchar[] unnest for string PKs, int[] for integer PKs
Concurrent writes to source during sync	Safe — single read-only source connection uses default_transaction_read_only=true. New transactions appear as higher xmin values; picked up on next pass

3.8 Latency characteristics (from README + source)

The README reports the following operational characteristics. Cold-start (full copy) latency is linear in row count and is NOT throttled. Incremental pass latency is throttled to the configured percentage. Delete scans are typically slower than update scans; SSLR explicitly supports splitting update-only and delete-only jobs so deletes can run less frequently.

Table 5: SSLR reported latency

Operation	Reported latency
Single-pass incremental sync (small table, ~1k rows)	~160ms (per README example, 2020-era hardware)
Read-progress reporting cadence (full copy)	Every 10,000 rows until 100k; every 100,000 rows until 1M; every 500,000 rows after
Recommended run frequency	Per README: "meant to be run regularly". No specific cadence enforced
Delete-scan cost on large tables	Documented as potentially slow; SSLR supports running deletes as a separate, less-frequent job

3.9 Behavioral contract for our PMS Adapter

The re-implementation in our TypeScript / Node.js adapter must satisfy the following runtime contract, derived from the above observations. These are the rules at runtime, not the source code that produces them:

- Read-only connection to source PMS database, with default_transaction_read_only=true enforced.

- Per-table xmin cursor persisted in our DB (the __sslr_state equivalent). Cursor advances only after a complete transaction has been consumed.
- Updates detected via xmin::text::bigint comparison; deletes detected via md5(array_agg(pk::varchar)) hash comparison on PK ranges.
- Divide-and-conquer delete detection with configurable chunk sizes and a floor at minDeleteChunkSize.
- Idempotent upserts: DELETE matching PKs + CopyFrom in a single transaction per chunk.
- Throttling via a 2× overshoot-sleep rule that targets a configurable source-DB utilization percentage.
- Schema-change handling: abort by default, full re-sync on opt-in.
- Filtered tables: per-table WHERE clause, validated against a declared "uses" list to prevent dependency cycles.
- No server-side extensions, no triggers on source, no logical replication slot. The contract is purely client-side.

4. Spec Card 2 — AWS Sample Hospitality Systems (Clean Guest API)

Source: github.com/aws-samples/sample-hospitality-systems · License: AWS Sample Code (modified Apache) · Stack observed: Python 3.12 Lambda + Aurora PostgreSQL + Cognito + API Gateway

Behavioral summary: a serverless hospitality platform with two REST APIs (CRS public, PMS staff). The guest API exposes six endpoints, returns a standard envelope on every response, enforces owner-only access on guest reads (CRS) and tenant-scoped access on list endpoints (PMS). The guest table is a single hub with linked spokes for reservations, loyalty, and identity documents.

4.1 Standard response envelope

Every response, success or error, uses this JSON envelope:

```
// Success { "success": true, "data": {...}, "metadata":
{...} }  // Error { "success": false, "error": { "code": "...",
"message": "...", "details": {...} } }
```

4.2 Guest API — six endpoints (runtime contract)

Table 6: AWS Sample Hospitality — guest API runtime contract

Method + path	Auth	Body	Returns	Tenant rule
POST /guests	Cognito ID token (caller's own sub)	{firstName, lastName, email, phone?, dateOfBirth?, preferences?, commPrefs?}	201 + guest object. If guest exists with same cognito_sub, backfills empty fields and returns 200	Caller's own sub only
GET /guests/{guestId}	Cognito ID token	—	200 + guest object (sensitive fields stripped); 404 if not found; 403 if guest.cognito_sub != caller.sub	Owner-only
PUT /guests/{guestId}	Cognito ID token	Partial guest fields	200 + updated guest object	Owner-only
GET /guests/{guestId}/reservations	Cognito ID token	?status=&page=&limit=	200 + paginated reservations joined with property_name, ordered by check_in DESC	Owner-only
GET /guests/{guestId}/loyalty	Cognito ID token	—	200 + {tier, pointsBalance, totalStays}	Owner-only
GET /guests?page=&limit= (PMS only)	Cognito ID token (staff group)	?page=&limit=	200 + paginated guest list, tenant-filtered by caller's property/region/chain scope	Property / region / chain

4.3 The guests table — schema reference

From migration 001_initial_schema.sql. This is the cleanest guest schema encountered in any of the five repositories. The hub-and-spoke pattern (guest record + linked reservations + linked loyalty + linked identity documents) is exactly what our own profile schema should look like.

Table 7: AWS Sample guests table — observed schema

Column	Type	Default / constraint	Purpose
guest_id	UUID	PRIMARY KEY DEFAULT gen_random_uuid()	Stable UUID PK; never reused

Column	Type	Default / constraint	Purpose
cognito_sub	VARCHAR	UNIQUE NOT NULL	JWT subject claim. Identity binding to auth provider
first_name	VARCHAR	Nullable	Given name
last_name	VARCHAR	Nullable	Family name
email	VARCHAR	UNIQUE NOT NULL	Primary contact email; uniqueness enforced at DB level
email_verified	BOOLEAN	DEFAULT false	Whether email ownership was confirmed
phone	VARCHAR	Nullable	E.164 preferred (no enforcement at DB level)
phone_verified	BOOLEAN	DEFAULT false	Phone ownership confirmation
date_of_birth	DATE	Nullable	ISO date
nationality	VARCHAR	Nullable	ISO 3166-1 alpha-2 country code
language	VARCHAR	DEFAULT 'en'	ISO 639-1 language code
address_line1..4	VARCHAR ×4	Nullable	Multi-line address (4 lines is generous for international)
city, state, province, county, postal_code, country	VARCHAR	Nullable	Address components, region-aware
preferences	JSONB	DEFAULT '{}'	Free-form JSONB for hotel-specific preferences
comm_prefs	JSONB	DEFAULT {email_opt_in: true, sms_opt_in: false, push_opt_in: false, marketing_opt_in: false}	Consent flags, with explicit defaults
stripe_customer_id	VARCHAR	Nullable	Payment integration reference
loyalty_id	UUID	Nullable	FK to loyalty profile
vip_level	VARCHAR	DEFAULT 'NONE' CHECK in (NONE, SILVER, GOLD, PLATINUM)	Loyalty tier with CHECK constraint
total_stays	INTEGER	DEFAULT 0	Denormalized count for fast display
total_spend	NUMERIC (12,2)	DEFAULT 0	Denormalized lifetime spend

Column	Type	Default / constraint	Purpose
last_stay_date	DATE	Nullable	Denormalized most-recent-stay date
tags	TEXT[]	Nullable	Postgres array of arbitrary tags
merged_from_ids	UUID[]	Nullable	Array of guest_ids that were merged into this one — provenance for dedup
created_at	TIMESTAMPTZ	DEFAULT NOW()	Creation timestamp
updated_at	TIMESTAMPTZ	DEFAULT NOW(), BEFORE UPDATE trigger	Auto-maintained on every update

Indexes: idx_guests_cognito_sub on cognito_sub (for auth-time lookup), idx_guests_email on email (for unique-constraint enforcement and search-by-email). The updated_at trigger is a standard BEFORE UPDATE that sets NEW.updated_at = NOW().

4.4 Sensitive-field stripping

GET /guests/{guestId} excludes three columns from the response even when the caller is the owner: identity_doc_type, identity_doc_number, identity_doc_expiry. These columns exist in the guest_identity_documents linked table and are read out via separate privileged flows only. The exclusion is enforced by a SENSITIVE_FIELDS constant in the handler, not at the DB level.

4.5 Tenant isolation — fail-closed by group claim

The most important behavioral pattern in this card. The PMS-side tenant isolation is enforced by a shared utility called resolve_property_scope(event) in src/layers/common/utils/tenant.py. Every PMS handler that lists guests (or stays, folios, tasks) calls this utility before issuing the SQL query. The contract is fail-closed — if the caller's Cognito groups do not establish a qualifying scope, the call is rejected, never widened.

Trust model (verbatim from the source):

> *Two claim families feed these decisions, and they are NOT equally trustworthy. cognito:groups is set only through the Cognito admin APIs (AdminAddUserToGroup) — a signed-in user cannot grant themselves a group. custom:property_id / custom:region are user-pool attributes, authoritative only because the user-pool clients do not list them in WriteAttributes. Access level is always derived from groups first, and the attribute claims only ever restrict further.*

Table 8: AWS Sample tenant-scope resolution rules

Caller has custom:property_id?	Caller's cognito:groups?	Resolved scope	Effect on list query
Yes (any group)	—	Property-scoped	Query filtered to that property_id only. Caller cannot widen or move properties
No	Admin / Manager / RevenueManager	Chain-level	Query returns all properties. Caller may voluntarily narrow via ?propertyId=
No	RegionalManager + has custom:region	Regional	Query filtered to region. May voluntarily narrow via ?propertyId=
No	RegionalManager + NO custom:region	DENIED	Rejected with ForbiddenError. Region claim missing = no scope granted
No	Any other group / no group	DENIED	Rejected. Never widened to chain

4.6 Behavioral contract for our Guest Profile API

- Single guests table with UUID PK, cognito_sub unique, email unique, JSONB preferences + comm_prefs, denormalized counts for fast display.
- Hub-and-spoke: GET /guests/{id} returns only the guest; reservations, loyalty, identity docs are linked sub-resources fetched on demand.
- Owner-only access on CRS endpoints (caller.sub == guest.cognito_sub); staff-only list endpoint on PMS, with tenant scope resolved via a single shared utility that fails closed when the group claim is absent.
- Standard response envelope on every endpoint: {success, data, metadata} or {success: false, error: {code, message, details}}.
- Sensitive fields (identity documents) stripped at the handler layer, not the DB layer; only fetched through separate privileged flows.
- Audit columns (created_at, updated_at) with a BEFORE UPDATE trigger that maintains updated_at automatically.

5. Spec Card 3 — KevAlien CRM (Hospitality State Machine)

Source: github.com/KevAlien/CRM · License: MIT + Commons Clause (NO COMMERCIAL CODE REUSE) · Stack observed: Python + LangGraph + FastAPI + Redis

Behavioral summary: a guest-state machine with 12 phases and an escalation side-state. Each guest's full journey is persisted as a JSON-serialized GuestState TypedDict in Redis, keyed by E.164-normalized phone number, with a 7-day TTL on inactivity. The WhatsApp channel, the LangGraph orchestration, and the local-LLM classifier are out of scope for our project; the state-machine vocabulary and the persistence pattern are what we re-implement.

5.1 The 12-phase guest lifecycle

Phases are declared as a Python Literal type in graph/state.py. The set is small, hospitality-specific, and vendor-neutral — none of the names reference WhatsApp, LangGraph, or Ollama. The same vocabulary will work for our receptionist screen regardless of which PMS we integrate with.

Table 9: KevAlien guest lifecycle phases

Phase	Meaning	Trigger to enter
UNKNOWN_CONTACT	First contact, no reservation	Session created
ACQUIRING	Collecting data for a booking	Guest starts asking about availability
BOOKING_RECEIVED	Booking created in PMS, proactive trigger fired	PMS booking webhook / scheduled poll
WELCOME_SENT	Welcome message sent to guest	T+0min after BOOKING_RECEIVED
INFO_SENT	Practical info (check-in time, parking, WiFi) sent	T+30min after BOOKING_RECEIVED
IDLE	Awaiting next lifecycle event	After INFO_SENT, until T-5d check-in
UPSELL	Upsell window (room upgrades, extra services)	T-5 days before check-in, only if phase=IDLE
PRE_CHECKIN	Arrival info + document collection	T-48h before check-in
CHECKIN_DAY	Day-of-arrival reminder	Check-in date
IN_HOUSE	Guest is on property	PMS check-in event
POST_STAY	Post-checkout review request	T+1 day after checkout
ESCALATED	Bot paused; staff notified (side state, reachable from any phase)	Complaint, out-of-scope request, or staff request

Recovery from ESCALATED: when staff resolves the issue, they manually reactivate the session. The session returns to IDLE (regardless of where it was when escalated). This is the only non-linear transition in the state machine.

5.2 Allowed transitions (observed in graph/builder.py)

Phase transitions are not arbitrary. The send_whatsapp_node function declares a transition table keyed by (current_phase, current_task) tuples. Three transitions are explicit; all others leave the phase unchanged.

Table 10: KevAlien phase transitions (task-driven)

Current phase	Triggering task	New phase
BOOKING_RECEIVED	simple_question	WELCOME_SENT
WELCOME_SENT	simple_question	INFO_SENT
INFO_SENT	simple_question	IDLE
Any phase	complaint	ESCALATED
Any phase	out_of_scope	ESCALATED
ESCALATED	manual staff reactivation	IDLE

5.3 Scheduled proactive triggers

The message_timeline.py scheduler declares time-based transitions independent of any guest message. These are driven by APScheduler with a Redis job store. Triggers are anchored to booking.checkin.

Table 11: KevAlien proactive timeline (anchored to booking)

Trigger time	Trigger type	Action
T+0min after booking received	WELCOME	Send welcome message
T+30min after booking received	PRACTICAL_INFO	Send check-in time, parking, WiFi info
T-5 days before check-in	UPSELL	Send upsell offer (only if phase=IDLE)
T-48h before check-in	PRE_CHECKIN	Ask arrival time, collect documents
Check-in day	CHECKIN_REMINDER	Send day-of-arrival reminder
T+1 day after checkout	POST_STAY	Send review request

5.4 Session persistence — Redis key + TTL

Sessions are JSON-serialized GuestState dicts, stored in Redis with the following contract:

- Key: hotel:session:{normalized_phone} where normalized_phone is E.164 without the + (e.g. +39 333 123456 → 39333123456; 0039… → 39…).
- Value: full GuestState JSON (utf-8, ensure_ascii=false).

- TTL: 7 days (604800 seconds). Refreshed on every save_session call.
- Conversation history capped at 50 entries; oldest entries dropped on overflow.
- last_interaction timestamp updated on every save.
- Fallback: if Redis is unreachable, save_session falls back to an in-memory dict (lost on process restart); load_session falls back to the same in-memory dict. The fallback logs an error but never throws.
- Connection pool: singleton, max_connections=20, decode_responses=true.

5.5 GuestState TypedDict — fields

Table 12: KevAlien GuestState fields

Field	Type	Purpose
guest	GuestInfo TypedDict	{phone, name?, language, is_known}
booking	BookingInfo TypedDict	{id?, checkin?, checkout?, room_type?, services[], num_guests?}
conversation_history	list<{role, content, timestamp}>	Capped at 50 entries
current_phase	PhaseType (Literal)	One of the 12 phases from Table 9
current_task	TaskType (Literal)	Classifier output: check_availability, analyze_needs, build_offer, simple_question, acquire_contact, complaint, out_of_scope
recommended_model	string	Ollama model name (out of scope for us)
pms_data	dict<string, any>	Local cache of raw PMS responses
offer	OfferOutput TypedDict	{offer_text, whatsapp_message, suggested_upsells[]}
pending_actions	list<dict<string, any>>	Actions awaiting staff review
bot_paused	bool (implicit)	Set true when ESCALATED; gates the router
escalation_reason	string (implicit)	Reason for the escalation
outbound_message	string (implicit)	The message to be sent next
last_interaction	ISO datetime (implicit)	Set on every save_session

5.6 Behavioral contract for our Guest Profile state

- Adopt the 12-phase vocabulary as our internal guest-phase enum. Derive phases from our reservations table, not pushed by the PMS.

- Allow transitions only via the explicit (current_phase, triggering_event) table. Anything not in the table leaves phase unchanged.
- Persist the full guest session as JSON in our DB (or Redis, if we add it later). Use the same hotel:session:{normalized_phone} key pattern.
- Cap conversation_history at 50 entries. This keeps the JSON payload bounded even for long-stay guests.
- Provide a graceful in-memory fallback when Redis is unavailable, but log loudly. The fallback is for dev/test only.
- Proactive timeline (T+0 WELCOME through T+1d POST_STAY) is a separate concern — scheduler-driven, not message-driven. We will not reproduce it in v1 but reserve the phase vocabulary for it.

6. Spec Card 4 — QloApps (Messy Source PMS)

Source: github.com/qloapps/qloapps · License: OSL-3.0 (core) / AFL-3.0 (modules) · Stack observed: PHP 8.2+ / MySQL 5.7+

Behavioral summary: a complete open-source hotel PMS in classic PHP server-rendered style. QloApps treats guests as a special case of customers: the PREFIX_customer table holds both registered customers and one-time guests (distinguished by is_guest=1). Guest search at runtime is a UNION of four LIKE queries against email, lastname, firstname, and id_customer. There is no fuzzy matching, no ranking, and no phone-based search in the public API. This is the representative shape of a legacy PMS our adapter will encounter in real hotels.

6.1 The PREFIX_customer table — what we'll be reading

QloApps uses a configurable table prefix (default ps_ but customizable). All tables follow the PREFIX_tablename convention. The customer table — which holds all guests — has the following schema (from install-dev/data/db_structure.sql):

Table 13: QloApps customer table (the messy source)

Column	Type	Notes for our adapter
id_customer	INT(10) UNSIGNED AUTO_INCREMENT	PK. Numeric, not UUID. Our adapter must map this to our UUID-based guest_id
id_shop_group	INT(11) UNSIGNED DEFAULT 1	Multi-tenant field at the schema level (QloApps is multi-shop)

Column	Type	Notes for our adapter
id_shop	INT(11) UNSIGNED DEFAULT 1	Specific shop within shop group
id_gender	INT(10) UNSIGNED NOT NULL	FK to gender table (Mr/Mrs/Ms/Dr etc.)
id_default_group	INT(10) UNSIGNED DEFAULT 1	Default customer group for pricing rules
id_lang	INT(10) UNSIGNED NULL	UI language preference
id_country	INT(10) DEFAULT 0	Default country code (FK)
id_risk	INT(10) UNSIGNED DEFAULT 1	Risk level (payment-fraud flag)
company	VARCHAR(64) NULL	Company name (if business guest)
siret	VARCHAR(14) NULL	French company registration number (QloApps is PrestaShop-derived, French-origin)
ape	VARCHAR(5) NULL	French NAF/APE business code
firstname	VARCHAR(32) NOT NULL	Given name. 32-char limit — important for our schema
lastname	VARCHAR(32) NOT NULL	Family name. 32-char limit
email	VARCHAR(128) NOT NULL	Email. 128-char limit. Uniqueness NOT enforced at DB level (no UNIQUE constraint)
passwd	VARCHAR(60) NOT NULL	Hashed password. We MUST NEVER read this column
last_passwd_gen	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	Internal
birthday	DATE NULL	ISO date
newsletter	TINYINT(1) UNSIGNED DEFAULT 0	Newsletter opt-in
ip_registration_newsletter	VARCHAR(15) NULL	IP at newsletter signup (legacy)
newsletter_date_add	DATETIME NULL	When newsletter opt-in was set
optin	TINYINT(1) UNSIGNED DEFAULT 0	Marketing opt-in (separate from newsletter)
website	VARCHAR(128) NULL	Personal website
outstanding_allow_amount	DECIMAL(20,6) DEFAULT 0.00	Credit allowance for invoiced payments

Column	Type	Notes for our adapter
show_public_prices	TINYINT(1) UNSIGNED DEFAULT 0	Whether this customer sees prices including tax
max_payment_days	INT(10) UNSIGNED DEFAULT 60	Max invoice payment terms
secure_key	VARCHAR(32) NOT NULL DEFAULT '-1'	Per-customer random token for cookie auth
note	TEXT NULL	Free-text internal staff note about this customer
phone	VARCHAR(32) NULL	Primary phone (NOT indexed, NOT searched by default)
active	TINYINT(1) UNSIGNED DEFAULT 0	Account active flag
is_guest	TINYINT(1) NOT NULL DEFAULT 0	1 = guest checkout (no account); 0 = registered customer
deleted	TINYINT(1) NOT NULL DEFAULT 0	Soft-delete flag. searchByName respects this
date_add	DATETIME NOT NULL	Creation timestamp
date_upd	DATETIME NOT NULL	Last update timestamp

Two indexes are declared: customer_email on email (used by login + uniqueness-via-application), and customer_login on (email, passwd) for the login flow. There is NO index on phone, NO index on firstname/lastname — meaning any search by those fields is a full table scan.

6.2 Guest search at runtime — the searchByName algorithm

From classes/Customer.php. The searchByName static method is the runtime contract for how QloApps (and PrestaShop-derived PMS systems in general) search for returning guests. The behavior we must match in our adapter's read-side transformation:

```
SELECT * FROM PREFIX_customer WHERE email LIKE '%{query}%'   [AND
deleted = 0]                  -- skip_deleted param   [+
Shop::addSqlRestriction]       -- multi-shop scoping UNION SELECT *
FROM PREFIX_customer WHERE id_customer = {query_as_int} UNION
SELECT * FROM PREFIX_customer WHERE lastname LIKE '%{query}%' UNION
SELECT * FROM PREFIX_customer WHERE firstname LIKE '%{query}%'
[LIMIT 0, {limit}]
```

Runtime behavior to capture: searchByName is a SUBSTRING match on email/firstname/lastname plus an exact integer match on id_customer, combined via UNION (so duplicates are de-duped by the SQL engine). There is no ranking — results come

back in the order the UNION produces them, which is non-deterministic across table statistics. The phone column is NOT searched. The deleted flag is respected only if skip_deleted=true is passed.

6.3 Webservice search — the API-level filter

From classes/webservice/WebserviceSpecificManagementBookings.php, line 3327. When the webservice is queried with filter[customer]={value}, the runtime SQL is:

```
AND (c.`firstname` LIKE "%{value}%"      OR c.`email` LIKE "%
{value}%")
```

Runtime behavior: the webservice search is narrower than searchByName — it searches only firstname and email, not lastname or id_customer. Phone is again not searchable. The LIKE patterns are double-quoted rather than single-quoted (a minor SQL injection surface that we do NOT need to reproduce — our adapter will use parameterized queries).

6.4 Guest-detail linkage — the PREFIX_guest table (separate)

QloApps has a SEPARATE PREFIX_guest table that is NOT the customer table. The guest table tracks anonymous browsing sessions (id_operating_system, id_web_browser, screen_resolution_x/y, javascript, mobile_theme, accept_language, etc.) and links to a customer via id_customer. This table is for analytics, not for guest profiles. Our adapter must read PREFIX_customer (the customer table) and ignore PREFIX_guest entirely.

6.5 Edge cases observed in the schema

Table 14: QloApps edge cases relevant to our adapter

Condition	Observed behavior / schema detail
Email uniqueness	NOT enforced at DB level. Application code checks via SELECT before INSERT. Multiple rows may exist with the same email over time (deleted=1 records are kept)
Guest vs registered user	Both in same table; is_guest=1 distinguishes. Guest records have no password, no secure_key chain. Both should be sync targets for our adapter
Soft delete	deleted=1 leaves the row in place. Our adapter must decide whether to keep or drop the corresponding profile. Recommendation: keep with a deleted_at timestamp
Multi-shop scoping	id_shop_group + id_shop. Our adapter's tenant filter is the equivalent — must scope reads to a single shop
French-specific fields	siret, ape are French business codes. Hotels outside France will have these NULL — our schema should not require them

Condition	Observed behavior / schema detail
Field length limits	firstname/lastname: 32 chars. email: 128 chars. phone: 32 chars. Our schema should accept these limits on ingress
Phone format	Not enforced. May be any string up to 32 chars. Our adapter must normalize to E.164
Birthday	Nullable DATE. May be 0000-00-00 (legacy MySQL strict-mode bypass). Our adapter must handle this and treat 0000-00-00 as null

6.6 Behavioral contract for our adapter's read-side transformation

- Source tables to read: PREFIX_customer (skip PREFIX_guest entirely).
- Source columns to read: id_customer, firstname, lastname, email, phone, birthday (treat 0000-00-00 as null), is_guest, deleted, date_add, date_upd.
- Source columns to NEVER read: passwd, secure_key, ip_registration_newsletter, last_passwd_gen. These are auth/security columns and would be a privacy violation.
- Mapping: id_customer (int) → source_ref (string) in our profile table. firstname + lastname → our name fields. email → email. phone → phone (normalized to E.164). birthday → date_of_birth. is_guest → a boolean flag we keep for analytics. deleted → soft-delete our profile with deleted_at timestamp.
- Search behavior: do NOT reproduce QloApps' UNION-of-LIKEs search. Our profile DB has its own search (Card 2's GET /guests endpoint). The adapter's job is to mirror data, not to mirror search.

7. Spec Card 5 — FormFilla (Smart Browser Form-Fill)

Source: github.com/AhmadHassan-BTed/FormFilla · License: MIT · Stack observed: TypeScript 5.4 / Chrome Manifest V3 extension

Behavioral summary: a Chrome extension that fills complex forms (job applications, but the algorithm is form-agnostic) using a three-stage pipeline: (1) scan and normalize form fields, (2) solve field-to-profile-key assignment as a maximum-weight bipartite matching via the Kuhn-Munkres (Hungarian) algorithm, (3) write values via native prototype setters to bypass React/Vue/Angular's synthetic event wrappers. 100% local-first, no network requests, sub-100ms execution. This is the runtime contract our browser-extension adapter must match when filling the hotel's existing browser-based registration form.

7.1 Three-stage pipeline (the architecture)

```
Active DOM  ⟶  1. Scanner & Normalizer  ⟶  2. Hungarian Solver
⟶  3. Framework Setter  ⟶  Filled Form
```

Stage 1 (Scanner & Normalizer): recursively walks the active DOM subtree (including same-origin iframes) and detects all input, select, textarea, and combobox elements. For each, it extracts five signals (label, placeholder, name, id, legend) and normalizes them by tokenizing against an abbreviation dictionary. The output is a list of FieldSignals objects, each carrying the five raw signals plus the element's autocomplete attribute, type, tagName, blockType (for repeated blocks like multiple work experiences), and blockIndex.

Stage 2 (Hungarian Solver): builds a cost matrix where rows are form fields and columns are candidate profile keys. Each cell's score is computed by the signal scorer (next section). The matrix is then solved by the Kuhn-Munkres algorithm for maximum-weight bipartite matching. The output is a globally-optimal assignment of profile keys to form fields, where no field is forced into a pairing unless it has a genuine candidate above the LOW_THRESHOLD.

Stage 3 (Framework Setter): writes the assigned values to the DOM using the native prototype setter bypass (section 7.4 below), then dispatches real input and change events so the host page's reactive framework (React/Vue/Angular) syncs its internal state.

7.2 Signal scorer — the scoring formula

From src/content/matching/scorer.ts. For each (form field, candidate profile key) pair, the score is computed as follows. Sources carry decreasing weights because labels are more reliable than placeholders, which are more reliable than id attributes:

Table 15: FormFilla signal sources and weights

Signal source	Weight	Why
label	1.0	Most reliable — set by the form author intentionally
placeholder	0.9	Slightly less reliable — sometimes auto-generated
name	0.8	Field name attribute; stable but technical
id	0.7	Field id; often contains framework prefixes
legend	0.6	Fieldset legend; least reliable

For each source, the scorer computes a sub-score against every rule in rules.json. The sub-score components, summed and multiplied by the source weight, then taken as the max across all sources:

- Exact phrase match (lowercase, after tokenization): +35 points
- Regex pattern match: +20 points

- Fuzzy similarity (max of Dice coefficient and Jaro-Winkler, against every exact phrase in the rule): if > 0.6, +max_similarity × 15 points
- Negative pattern match (rule declares patterns that should disqualify the field): −40 points
- Autocomplete token match (HTML autocomplete attribute matches a rule.autocompleteTokens entry): +45 points (applied once, not per source)
- Block context bonus (field is in a repeated block whose type matches the rule's expected block type): +15 points (applied once)

A candidate is included in the cost matrix only if its final score >= LOW_THRESHOLD. Candidates above HIGH_THRESHOLD are confirmed without user review; candidates between LOW_THRESHOLD and HIGH_THRESHOLD are highlighted amber for receptionist confirmation.

7.3 The Kuhn-Munkres solver — global optimality

From src/content/matching/assignment.ts. The matching algorithm treats the assignment as a minimum-cost perfect matching problem on a bipartite graph, with cost = (max_score − field score). The matrix is padded to square with a uniform "no relationship" sentinel cost so that fields with no genuine candidates do not steal keys from deserving fields. The solver is $O(dim^3)$ where dim = max(fields.length, distinct key count). For realistic form sizes (tens to low hundreds of fields), execution is single-digit to low double-digit milliseconds; a 500-field × 200-key stress test still completes in well under a second.

Critical correctness property (from the source code comments, which we trust more than any external documentation):

> *Because every non-candidate cell (a genuine "field X has nothing to do with key Y", or pure structural padding) costs exactly the same, total cost over any perfect matching always decomposes as (matrix size) × (sentinel) − (sum of scores used). Minimizing that is identical to maximizing total matched score, so a field with a genuine candidate is never displaced by one that lacks one.*

Defensive behaviors in the solver: non-finite scores (NaN, Infinity) are dropped before matrix construction; repeated (field, key) pairs collapse to their best score; the final output is de-duplicated so a malformed upstream candidate list degrades gracefully instead of propagating garbage.

7.4 Native prototype setter — the framework-safe write

From src/content/fill/native-setter.ts. Plain element.value = x assignments fail silently on React/Vue/Angular pages because the framework installs a value tracker on the instance that marks the assignment as already-seen, suppressing the onChange callback. FormFilla bypasses this by calling the value setter defined on the element's PROTOTYPE directly (via Function.prototype.call), which the framework's instance-level wrapper cannot intercept.

After the value is set, two events are dispatched to match what a real user edit produces:

- Event 1: input (bubbles=true, composed=true). The inputType defaults to 'insertReplacementText' — the standard value for a single-operation content replacement (not 'insertFromPaste').
- Event 2: change (bubbles=true, composed=true). Real typing fires change on blur/commit, and so does this.

Defense in depth: if the page has somehow replaced the prototype's own value descriptor (rare — aggressive polyfill or anti-automation script), the prototype lookup fails silently and falls back to plain el.value = x. A second fallback uses a hidden same-origin about:blank iframe's untouched copy of the same built-in prototype. Blank iframes get a fully independent set of global constructors synchronously on attach, which is the standard technique for recovering an unpatched native reference.

7.5 Dictionary structure — rules.json

FormFilla's matching logic is decoupled from code via four externalized JSON dictionaries. The rules.json file is the most important — each key is a profile JSON path (e.g. personal.firstName, documents.passport) and each value is a rule with these fields:

Table 16: FormFilla rule structure

Rule field	Type	Purpose
exactPhrases	string[]	Lowercase literal strings; +35 points if any matches the tokenized source
regexPatterns	string[]	Case-insensitive regex strings (no enclosing slashes); +20 points if any matches
autocompleteTokens	string[]	HTML autocomplete attribute values; +45 points if matched
allowedTypes	string[]?	Constrains the rule to specific input types (e.g. ['file'] for documents, ['email'] for email fields)
negativePatterns	regex[]?	Patterns that disqualify the field; −40 points if any matches

Rule field	Type	Purpose
sensitivityTier	'standard'|'sensitive'|'critical'?	Critical fields trigger verification overlays before fill
requiresOptIn	boolean?	If true, the rule is gated behind a dedicated user-facing privacy toggle
optInKey	string?	The profile.optIns key that gates this rule (e.g. 'sensitiveEeo', 'sensitiveCompensation')

7.6 Latency characteristics

Table 17: FormFilla reported latency

Operation	Reported latency
End-to-end fill (typical job application form, ~50 fields)	<100ms (per README badge)
Hungarian solver, realistic form size	Single-digit to low double-digit ms
Hungarian solver, stress test (500 fields × 200 keys)	<1 second
Network requests during fill	Zero (100% local-first)

7.7 Behavioral contract for our browser-extension adapter

- Three-stage pipeline: scan → score → solve → set. No shortcuts.
- Five signal sources per field (label, placeholder, name, id, legend), with weights 1.0/0.9/0.8/0.7/0.6. Take the max across sources.
- Scoring: exact phrase +35, regex +20, fuzzy similarity (max Dice + Jaro-Winkler) × 15 if > 0.6, autocomplete +45, block context +15, negative −40. LOW_THRESHOLD and HIGH_THRESHOLD control confirmation flow.
- Solver: Kuhn-Munkres on cost matrix (max_score − score). Pad to square with uniform sentinel. $O(dim^3)$.
- Native prototype setter bypass for React/Vue/Angular. Fallback chain: prototype setter → el.value = → pristine-iframe prototype.
- Dispatch input + change events with bubbles=true, composed=true after every value write. Use inputType='insertReplacementText'.
- Rules are externalized JSON (rules.json equivalent) so we can extend for hotel-specific registration forms without code changes.
- Receptionist confirmation flow: high-confidence matches fill silently; medium-confidence matches highlight amber and wait for click; nothing below LOW_THRESHOLD fills.

8. Cross-Cutting Synthesis — What the Five Cards Tell Us Together

8.1 The adapter is two distinct runtime contracts

Card 1 (SSLR) and Card 5 (FormFilla) together describe the two halves of the PMS Adapter — and they have nothing in common except the name. SSLR is a backend sync primitive: stateful, throttled, background, eventually-consistent, batched. FormFilla is a frontend DOM-write primitive: stateless per fill, instant, interactive, transactional. Combining them into one component would couple two unrelated concerns. The Step #3 PM planning must split them as Adapter-Read and Adapter-Fill, with a shared guest-data contract between them.

8.2 Our schema is Card 2's guests table, with Card 3's phase enum

Card 2's guests table (Section 4.3) is the cleanest guest schema encountered. Card 3's 12-phase enum (Section 5.1) is the cleanest hospitality-specific lifecycle vocabulary. The Step #4 OpenSpec design should adopt both: UUID PK, cognito_sub unique, email unique, JSONB preferences and comm_prefs, denormalized counts, hub-and-spoke linked resources, and the 12-phase enum as a guest_phase column. We add a source_ref column (string) so the adapter can map our profile back to the source PMS's customer id.

8.3 Card 4 is what real source data looks like

Card 4's QloApps schema is the representative shape of legacy PMS data we will encounter in the wild. INT PKs (not UUIDs), French-specific fields (siret, ape), 32-char name limits, no email uniqueness at the DB level, soft-delete flags, and a separate PREFIX_guest table that we must explicitly NOT read. Our adapter's read-side transformation (Card 1's contract applied to Card 4's source shape) must handle all of these gracefully.

8.4 Fail-closed tenant isolation is non-negotiable

Card 2's resolve_property_scope utility (Section 4.5) is the correct pattern. The rule is: derive access level from a signed group claim first; let attribute claims only narrow further; never widen. Absence of a property claim is NOT proof of chain-level access. Even though our v1 is single-tenant, building this boundary in from day one is materially cheaper than retrofitting it. Every list endpoint in our profile API must call a single shared tenant-scope resolver before issuing the SQL.

8.5 The receptionist screen's job is small and bounded

Combining Card 2's hub-and-spoke pattern (small GET /guests/{id} payload) with Card 5's sub-100ms form-fill (Card 5 Section 7.6), the receptionist screen has a tight runtime budget: type a few characters, get a list of matches in <300ms, click one, get the profile in <100ms, click "Fill registration form," get the form filled in <100ms. Total round-trip budget under 500ms. The Card 2 design (small payloads, linked sub-resources) and the Card 5 design (sub-100ms solver) are what make this budget achievable.

8.6 What we explicitly do NOT inherit

- SSLR's Go-specific pgx driver usage — we re-implement in TypeScript with node-postgres or postgres.js.
- AWS Sample's AWS Lambda + Aurora + Cognito + API Gateway stack — we re-implement in Node.js + SQLite/Postgres + JWT.
- KevAlien's LangGraph + Ollama + WhatsApp stack — we keep only the state machine vocabulary and the Redis persistence pattern.
- QloApps' PHP + MySQL + PrestaShop-derivative stack — we keep only the schema-shape knowledge for adapter read-side mapping.
- FormFilla's job-application rules.json — we keep the algorithm and write our own hospitality-specific rules dictionary.

9. Hand-off to Step #3 (Product-Manager Planning)

This document is the sole input to the Product-Manager planning step. The PM agent's CIRCLES method (Comprehend → Identify → Report → Cut → List → Evaluate → Summarize) should consume these five cards as the discovery corpus, layer in our project constraints (read-only source access, one-way sync, single-developer maintainable, NDPR/GDPR/CCPA compliance, no AI features, no automatic dedup), and produce a phased build plan sequenced against the Blueprint skill's 80-area checklist.

Recommended prioritization for the PM step:

- Quick win (Phase 0): Card 2's guests table schema as our initial DB schema. Card 3's phase enum as our initial guest_phase column. Both are pre-decisions, ready to ship.
- Core bet (Phase 1): Card 1's read-side sync contract, implemented in TypeScript against node-postgres, with Card 4's QloApps schema as the first source shape. End state: data flows from a real QloApps install into our profile DB.

- Core bet (Phase 2): Card 2's guest API (six endpoints, standard envelope, owner-only + tenant-scoped). Card 5's frontend form-fill as a separate browser extension.
- Longer horizon (Phase 3): additional adapter shapes (API-based PMS like Mews/Cloudbeds; Windows desktop PMS via a companion app). Reserve the Card 3 proactive timeline for v2.

Open questions for the PM step to resolve:

- Do we use SQLite or Postgres for our target DB? Postgres is the natural fit given Card 1's pgx-native contract; SQLite simplifies single-hotel deployments.
- Do we adopt Card 2's envelope exactly, or simplify? The standard envelope adds friction on every handler; for a single-dev project, a simpler success/failure shape may be preferable.
- How do we authenticate receptionists in v1? Card 2 uses Cognito; we may prefer a simpler JWT-with-group-claims scheme.
- For the browser-extension adapter (Card 5), do we ship Manifest V3 (Chrome-only) or also support Firefox? V3 is required for Chrome Web Store; Firefox supports V3 with caveats.

These questions are for Step #3 to resolve. Step #4 (OpenSpec) will then formalize the chosen answers as delta specs against the project's main specs, and the build will execute against those specs.